Here's a **clear, interview-grade breakdown** of the major JVM garbage collectors, focusing on *how they actually work internally*, not just definitions.

🧠 Big Picture First

All modern JVM GCs are built on:

- **Generational hypothesis** (young vs old objects)
- **Tri-color marking**
- Different trade-offs between:
  - ⏱️ Latency (pause time)
  - ⚡ Throughput
  - 🧩 Complexity

1️⃣ Serial GC (Single-threaded, Stop-the-world)

💡 Idea:

- Simplest GC
- Uses **one thread**
- Entire application pauses during GC

🔄 How it works

Young Generation (Minor GC)

- Uses **Copying (Mark-Copy)**
- Eden → Survivor spaces

```
Eden → copy live → Survivor
```

Old Generation (Major GC)

- Uses **Mark-Sweep-Compact**

Steps:

1. Mark reachable objects
2. Sweep garbage
3. Compact memory (remove fragmentation)

⚙️ Characteristics

- ❌ Long pauses
- ✅ Simple and predictable
- ✅ Good for small heaps / single-core

2️⃣ Parallel GC (Throughput GC)

💡 Idea:

- Same algorithm as Serial
- But uses **multiple threads**

🔄 How it works

Young Gen:

- Parallel **copying GC**

Old Gen:

- Parallel **mark-sweep-compact**

⚙️ Characteristics

- ✅ High throughput
- ❌ Still **Stop-the-World (STW)**

- ❌ Not latency-friendly

🧠 Key difference vs Serial

Feature	Serial	Parallel
Threads	1	Multiple
Pause	STW	STW
Speed	Slow	Faster

3️⃣ CMS (Concurrent Mark Sweep)

💡 Idea:

Reduce pause times by doing most work **concurrently**

🔄 Phases

1. Initial Mark (STW, short)

- Mark root objects

---

2. Concurrent Mark

- Traverse object graph (tri-color marking)

---

3. Remark (STW)

- Fix mutations (using write barriers)

---

4. Concurrent Sweep

- Remove garbage

⚠️ No Compaction!

- Memory becomes fragmented
- Leads to:
  - Allocation failures
  - Full GC fallback

⚙️ Characteristics

- ✅ Low pause times
- ❌ Fragmentation problem
- ❌ Complex tuning
- ❌ Deprecated (removed in newer JVMs)

4️⃣ G1 GC (Garbage First)

💡 Idea:

- Heap divided into **regions (not contiguous generations)**
- Collect **most garbage first**

🧩 Heap Structure

```
Heap → Regions (1–32 MB each)
```

```
Types:
- Eden
- Survivor
- Old
- Humongous
```

🔄 Phases

1. Young GC (STW)

- Copy live objects (like Parallel GC)

---

2. Concurrent Mark

- Global marking (tri-color)

---

3. Remark (STW)

- Fix mutations

---

4. Cleanup

- Identify garbage-heavy regions

---

5. Evacuation (STW)

- Copy live objects from selected regions

🎯 Key Innovation

👉 **Region-based collection**

- Doesn't collect entire heap
- Picks **regions with most garbage**

⚙️ Characteristics

- ✅ Predictable pause times
- ✅ Compaction (no fragmentation)
- ✅ Default GC in modern JVMs
- ❌ Slight overhead

5️⃣ Shenandoah GC (Ultra-low latency)

💡 Idea:

- **Almost no pause times**
- Move objects **while program is running**

🔄 How it works

Key concept:

👉 **Concurrent Compaction**

Phases:

1. Initial Mark (STW, tiny)
2. Concurrent Mark
3. Concurrent Evacuation (🔥 unique)
4. Concurrent Update References
5. Final Update (very small STW)

🧠 Trick: Brooks Pointer

Each object has an **indirection pointer**:

```
Object → forwarding pointer → actual location
```

👉 Allows moving objects safely during execution

⚙️ Characteristics

- ✅ Very low pause times (<10ms)
- ✅ No fragmentation
- ❌ Higher CPU overhead
- ❌ More complex

6️⃣ ZGC (Bonus – modern alternative)

💡 Idea:

- Region-based + colored pointers
- Fully concurrent GC

🧠 Key Innovation

👉 **Colored pointers**

- Metadata embedded in pointers
- Track object state without stopping threads

⚙️ Characteristics

- ✅ Sub-10ms pauses (even with huge heaps)
- ✅ Scales to TB heaps
- ❌ Requires newer JVM + hardware support

🔥 Comparison Summary

GC	Threads	Pause Time	Compaction	Fragmentation	Best For
Serial	1	High	Yes	No	Small apps
Parallel	Many	High	Yes	No	Throughput
CMS	Many	Low	No	Yes ❌	Legacy low-latency
G1	Many	Medium	Yes	No	Balanced (default)
Shenandoah	Many	Very Low	Yes (concurrent)	No	Low-latency
ZGC	Many	Ultra Low	Yes (concurrent)	No	Large heaps

🧠 Mental Model (Interview)

Think evolution:

```
Serial → Parallel → CMS → G1 → Shenandoah/ZGC
(simple → concurrent → region-based → fully concurrent)
```

🔥 Interview One-Liner

> "Modern GCs like G1, Shenandoah, and ZGC use tri-color marking with concurrent phases and write barriers, but differ in how aggressively they reduce pause times and handle compaction."

If you want next-level depth, I can explain:

- **SATB vs Incremental Update (G1 vs CMS)**
- **Exact write barriers used in each GC**
- **When to choose which GC in production (real scenarios)**